

**Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

ОТЧЁТ
по задачам: I-L
по дисциплине «Алгоритмы и структуры данных»

Выполнил:
Ануфриев Андрей Сергеевич
Р3219

Содержание

1	Задание I. Машинки	2
2	Задание J. Гоблины и очереди	4
3	Задание K. Менеджер памяти-1	6
4	Задание L. Минимум на отрезке	9

1 Задание I. Машинки

```
#include <algorithm>
#include <iostream>
#include <set>
#include <vector>

using namespace std;

int main() {
    int N, K, P;
    cin >> N >> K >> P;
    vector<int> seq(P);
    for (int i = 0; i < P; i++) {
        cin >> seq[i];
        seq[i]--;
    }
    vector<int> next_pos(P, P + 1);
    vector<int> last_occurrence(N, P + 1);
    for (int i = P - 1; i >= 0; i--) {
        int car = seq[i];
        next_pos[i] = last_occurrence[car];
        last_occurrence[car] = i;
    }

    set<pair<int, int> > on_floor;
    vector<bool> on_floor_flag(N, false);
    vector<int> current_next(N, P + 1);
    int operations = 0;

    for (int i = 0; i < P; i++) {
        int car = seq[i];

        if (on_floor_flag[car]) {
            on_floor.erase({current_next[car], car});
            current_next[car] = next_pos[i];
            on_floor.insert({current_next[car], car});
        } else {
            operations++;

            if ((int) on_floor.size() == K) {
                auto it = prev(on_floor.end());
                int removed_car = it->second;
                on_floor.erase(it);
                on_floor_flag[removed_car] = false;
                current_next[removed_car] = P + 1;
            }

            current_next[car] = next_pos[i];
            on_floor.insert({current_next[car], car});
            on_floor_flag[car] = true;
        }
    }

    cout << operations << endl;
```

```
return 0;  
}
```

Суть решения

Ключевая идея: оптимальная кеш-стратегия - когда пол переполнен, убрать машинку, которая будет запрошена позже всех (или вообще больше не понадобится). Для этого предварительно считаем для каждой позиции, когда машинка потребуется в следующий раз.

Основной алгоритм:

1. **Предвычисление next_pos:** проходим последовательность запросов в обратном порядке. Для каждой позиции i считаем $next_pos[i]$ - позицию следующего вхождения этой машинки в будущем. $last_occurrence$ хранит последнее (т.е. ближайшее в будущем) вхождение каждой машинки.
2. **Поддержание машинок на полу:** используем $set<pair<int, int>$ на $полу$ с ключом (текущее_время_следующего_вхождения, номер_машинки). Set автоматически держит машинку с наибольшим временем следующего вхождения.
3. **Обработка каждого запроса i :**
 - Если машинка уже на полу: удаляем её из set со старым ключом и добавляем с новым (время следующего вхождения из $next_pos[i]$).
 - Если машинки нет на полу:
 - Увеличиваем счётчик операций.
 - Если пол переполнен, удаляем из конца set машинку, которая будет запрошена позже всех.
 - Добавляем новую машинку с её временем следующего вхождения из $next_pos[i]$.

По сути я просто реализовал Belady's optimal algorithm для данной задачи.

Время: $O(P \log K)$ - P запросов, для каждого одна операция с set размером до K .

Память: $O(N + P)$ - массивы $next_pos$ и on_floor , последовательность запросов.

2 Задание J. Гоблины и очереди

```
#include <deque>
#include <iostream>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    cin >> N;

    deque<int> left, right;

    for (int i = 0; i < N; i++) {
        char op;
        cin >> op;

        if (op == '+') {
            int x;
            cin >> x;
            right.push_back(x);

            if (right.size() > left.size()) {
                left.push_back(right.front());
                right.pop_front();
            }
        } else if (op == '*') {
            int x;
            cin >> x;

            if (left.size() > right.size()) {
                right.push_front(x);
            } else {
                left.push_back(x);
            }

            if (left.size() > right.size() + 1) {
                right.push_front(left.back());
                left.pop_back();
            }
        } else if (op == '-') {
            cout << left.front() << '\n';
            left.pop_front();

            if (left.size() < right.size()) {
                left.push_back(right.front());
                right.pop_front();
            }
        }
    }

    return 0;
}
```

Суть решения

Ключевая идея: разделить очередь на две половины с помощью двух деков, поддерживая инвариант $\text{left.size()} \geq \text{right.size()}$. Это позволяет вставлять гоблинов в условную середину за $O(1)$ за счёт операций переноса элементов между деками.

Основной алгоритм:

1. **Инвариант структуры:** поддерживаем два дека (left и right). Начало очереди - front левого дека.
2. **Операция $+$ x:**
 - Добавляем x в конец right.
 - Проверяем баланс: если right стал больше left, то переносим элемент из front right в back left, чтобы восстановить инвариант.
3. **Операция $*$ x:**
 - Если $\text{left.size()} > \text{right.size()}$ (нечётная длина очереди), добавляем x в front right - это ровно за центром.
 - Если $\text{left.size()} == \text{right.size()}$ (чётная длина), добавляем x в back left - это сразу в центр.
4. **Операция $-$:**
 - Выводим front left (первого гоблина) и удаляем его.
 - Проверяем баланс.

Время: $O(N)$ - N операций, каждая за $O(1)$.

Память: $O(N)$ - два дека суммарно содержат N элементов.

3 Задание К. Менеджер памяти-1

```
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <map>
#include <random>
#include <set>

using namespace std;
typedef int ll;
typedef long double ld;
typedef string str;

void f() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
}

int main() {
    f();
    ll n, m;
    cin >> n >> m;
    set<pair<ll, ll> > a;
    map<ll, ll> b;
    map<ll, ll> c;
    map<ll, pair<ll, ll> > q;
    a.insert({n, 0});
    b[0] = n;
    c[n - 1] = n;
    for (ll s = 1; s <= m; ++s) {
        ll x;
        cin >> x;
        if (x > 0) {
            if (ll(a.size()) == 0) {
                cout << -1 << "\n";
                q[s] = {-1, -1};
                continue;
            }
            ll z = a.rbegin()->first, y = a.rbegin()->second;
            if (z == x) {
                cout << y + 1 << "\n";
                q[s] = {z, y};
                a.erase({z, y});
                b.erase(y);
                c.erase(y + z - 1);
            } else if (z > x) {
                cout << y + 1 << "\n";
                q[s] = {x, y};
                a.erase({z, y});
                b.erase(y);
                c.erase(y + z - 1);
                a.insert({z - x, y + x});
            }
        }
    }
}
```

```

        b[y + x] = z - x;
        c[y + z - 1] = z - x;
    } else {
        cout << -1 << "\n";
        q[s] = {-1, -1};
    }
} else {
    x = -x;
    auto v = q[x];
    if (v.first == -1 && v.second == -1) {
        continue;
    } else {
        if (c.count(v.second - 1) > 0) {
            pair<ll, ll> z = {v.second - 1, c[v.second - 1]};
            c.erase(v.second - 1);
            a.erase({z.second, z.first - z.second + 1});
            b.erase(z.first - z.second + 1);
            v.first += z.second;
            v.second = z.first - z.second + 1;
        }
        if (b.count(v.second + v.first) > 0) {
            pair<ll, ll> y = {v.second + v.first, b[v.second + v.first]};
            b.erase(v.second + v.first);
            a.erase({y.second, y.first});
            c.erase(y.first + y.second - 1);
            v.first += y.second;
        }
        a.insert(v);
        b[v.second] = v.first;
        c[v.second + v.first - 1] = v.first;
    }
}
}
return 0;
}

```

Суть решения

Ключевая идея: представить память как множество свободных блоков (размер, начало). При выделении найти первый подходящий блок, при освобождении слить освобождённый блок с соседними свободными блоками.

Основной алгоритм:

1. **Инициализация:** начально вся память свободна - добавляем в set a блок (размер N, начало 0). Так же используем maps b и c для быстрого поиска соседей по началу и концу блока.
2. **Запрос выделения K ячеек:**
 - Получаем наибольший свободный блок из set rbegin().
 - Если его размер z равен K, выделяем весь блок.
 - Если $z > K$, выделяем первые K ячеек начиная с позиции y, а остаток (z-K ячеек начиная с y+K) остаётся свободным.
 - Если $z < K$, запрос отклоняется - выводим -1.

3. Запрос освобождения памяти:

- Если запрос T был отклонен, игнорируем.
 - Проверяем наличие блока слева, заканчивающегося в позиции $v.second - 1$. Если есть, объединяем его с текущим.
 - Проверяем наличие блока справа, начинающегося в позиции $v.second + v.first$. Если есть, объединяем его с текущим.
- Добавляем объединённый блок обратно.

Тут у меня было много попыток оптимизировать время. В итоге нашёл эту задачу в интернете и посмотрел на неё разбор. Написал примерно такое-же решение.

Время: $O(M \log M)$ - M запросов, для каждого одна-две операции с `set`.

Память: $O(M)$ - хранение информации о выделенных блоках и свободных сегментах.

4 Задание L. Минимум на отрезке

```
#include <deque>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int N, K;
    cin >> N >> K;
    vector<int> arr(N);
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
    }
    deque<int> dq;
    for (int i = 0; i < N; i++) {
        if (!dq.empty() && dq.front() <= i - K) {
            dq.pop_front();
        }
        while (!dq.empty() && arr[dq.back()] >= arr[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        if (i >= K - 1) {
            cout << arr[dq.front()];
            if (i < N - 1) {
                cout << " ";
            }
        }
    }
    cout << "\n";
    return 0;
}
```

Суть решения

Ключевая идея: использовать двустороннюю очередь для эффективного поддержания минимума в скользящем окне. Дек хранит индексы элементов текущего окна, упорядоченные по значениям в возрастающем порядке.

Основной алгоритм:

1. **Инвариант дека:** дек содержит индексы элементов, находящихся в текущем окне, в порядке возрастания их значений. Минимум текущего окна всегда находится в front дека.
2. **Удаление устаревших индексов:**
 - Перед обработкой элемента i проверяем, не устарел ли индекс в front дека.
 - Условие: $\text{dq.front()} \leq i - K$ означает, что индекс вышел за левую границу окна $[i - K + 1, i]$.
 - Если устарел, удаляем его из front дека.
3. **Добавление нового элемента:** добавляем индекс i в конец дека. Теперь дек снова упорядочен по возрастанию значений.

Изначально хотел сделать 2 указателя на массиве, но это было бы $O(N \cdot K)$ в худшем случае. Поэтому взял просто двустороннюю очередь.

Время: $O(N)$ - N элементов, каждый обрабатывается константное число раз.

Память: $O(K)$ - дек содержит не более K элементов (размер окна).